

國立暨南國際大學資訊工程學系

碩士論文

A*演算法在片段限制最長共子序問題之應用
Application of the A* Algorithm to Solve the Longest
Common Subsequence from Fragments Problem

指導教授：李家同教授

研究生：潘鈺梅

中華民國九十四年六月

Acknowledgement

As my thesis is finally getting completed, I am very grateful for my dear advisor - Professor Richard Chia-Tung Lee for his long-term teaching. During the two years in NCNU, he has not only taught me in doing research but also guided me to know a lot of things in the world. Without his teaching, I can't complete my thesis so smoothly. I would like to thank Prof. Lee for paying a lot of attention and concern to me.

I also like to thank Professor Tung-Shou Chen, Professor Chwei-Shyong Tsai and Professor Hsien-Chu Wu in NTIT. Without their teaching and encouragement, I can't enter NCNU, and get the master's degree.

In the meanwhile, I would like to thank Professor Chien-Hung Huang in NFU. During the finalization of the thesis, he gave me many valuable suggestions. I also would like to thank Professor Guan-Shieng Huang and my group members and classmates for giving me much assistance and encouragement. Finally, I like to thank my dear parents. I can smoothly complete my master's degree due to their spiritual support and fosterage. Thank you!

Yu-Mei Pan 潘鈺梅

Computer Science and Information Engineering Department

National Chi-Nan University

Puli, Nantou, Taiwan

June, 2005

論文名稱：A*演算法在片段限制最長共子序問題之應用

校院系：國立暨南國際大學 資訊工程學系

頁數：三十八

畢業時間：九十四年六月

學位別：碩士

研究生：潘鈺梅

指導教授：李家同

摘要

在生物資訊的領域中，最長共子序問題是一個非常重要的問題，而這個問題的定義如下：給兩條序列 X 和 Y ，找一條 X 和 Y 的共同子序列，而這個共同子序列的長度是最長的。在本篇論文中，我們則是要討論一個最長共子序問題的一個衍生問題——片段限制最長共子序問題，而問題的定義如下：給兩條序列 X 和 Y ，還有一個 X 和 Y 的共同序列片斷的集合 M ，利用集合 M 的資訊去找出一條 X 和 Y 的共同子序列，而找到的這個共同子序列的長度是最長的。本篇論文中，我們應用樹狀搜尋策略演算法中的——A*演算法去解這一個片段限制最長共子序的問題。在我們所提出的 A*演算法，它可以幫助我們去除掉一些不會在結果中出現的共同序列片斷。藉由刪除一些不必要的共同序列片斷，我們則能夠更有效率地去找答案。然而，在最差的情形之下，所有給定的共同序列片斷仍需要全部被考慮、計算才能找到答案。

關鍵字：最長共子序、共同序列片斷、樹狀搜尋策略、A*演算法

Title of Thesis : Application of the A^* Algorithm to solve the Longest Common Subsequence from Fragments Problem

Name of Institute : CSIE, NCNU

Pages : 38

Graduation Time : 06/2005

Degree Conferred : Master

Student Name : Y. M. Pan

Advisor Name : Chia-Tung Lee

Abstract

Finding longest common subsequence (LCS) is a common problem in Biology informatics. The problem is defined as follows: Given two strings $X=x_1x_2\dots x_m$ and $Y=y_1y_2\dots y_n$, find a common subsequence $L=l_1l_2\dots l_p$ of X and Y such that p is maximized. In this thesis, we discuss a variation of the LCS problem – LCS from fragments problem defined as follows: Given two strings X and Y and a set M of fragments which are matching substrings of X and Y , find a LCS from M . A new method using a tree searching strategy, A^* algorithm, is proposed in this study for the LCS from fragments problem. The method can help us to filter out some fragments which wouldn't appear in solutions, and efficiently find a solution. However, in worst cases, all fragments are needed to be computed in the solving process.

Keywords: longest common subsequence, fragment, tree searching strategy, A^* algorithm

Contents

List of Figures.....	v
List of Tables	vii
Chapter 1 Introduction.....	1
1.1 The Edit Distance	3
1.2 The Edit Graph Shortest Path Problem.....	3
Chapter 2 A Dynamic Programming Approach.....	7
2.1 Baker-Giancarlo Approach	7
2.2 An Example	12
Chapter 3 The A* Algorithm	17
3.1 The Algorithm	24
3.2 Examples	28
Chapter 4 Experiments	32
Chapter 5 Conclusions.....	35
Bibliography	36

List of Figures

Fig. 1-1	The edit graph of two strings $X=tacat$ and $Y=actat$	4
Fig. 1-2	The edit graph of two strings $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$	5
Fig. 1-3	The shortest path of two strings $X=tacat$ and $Y=actat$ and the set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$	6
Fig. 2-1	An example of $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$. (a) The edit graph of X and Y and the set M . (b) The shortest path of X and Y and the set M	8
Fig. 2-2	Three regions of fragment f	10
Fig. 2-3	The relations between fragments $f_1 \sim f_8$ and f'	10
Fig. 2-4	The edit graph of two strings $X=atcgatcgc$ and $Y=ccgtccgac$ and a set $M=\{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}$	13
Fig. 2-5	The values in the nodes are the cost of the set $M=\{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}$ and the cost of node (m, n)	14
Fig. 2-6	The shortest path of two strings $X=atcgatcgc$ and $Y=ccgtccgac$ and the set $M=\{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}$	15
Fig. 3-1	The edit graph of two strings $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$	17
Fig. 3-2	Node $(0, 0)$ selected to expand of Fig. 3-1.....	18
Fig. 3-3	$f_{(2, 1, 2)}$ selected to expand of Fig. 3-1.....	19

Fig. 3-4	$f_{(4, 4, 2)}$ selected to expand of Fig. 3-1.....	20
Fig. 3-5	Node (m, n) selected to expand of Fig. 3-1.....	20
Fig. 3-6	The shortest path of two strings $X=tacat$ and $Y=actat$ and $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$	21
Fig. 3-7	p is a node of the expanded tree.....	22
Fig. 3-8	The edit graph of two strings $X=atcgatcgc$ and $Y=ccgtccgac$ and a set $M=\{f_{(2, 4, 2)}, f_{(3, 2, 2)}, f_{(3, 6, 2)}, f_{(5, 7, 2)}, f_{(7, 4, 2)}, f_{(8, 2, 2)}, f_{(8, 6, 2)}\}$	23
Fig. 3-9	The diagonal line of edit graph.....	26
Fig. 3-10	Given fragments distributed over the diagonal line of the edit graph where the dotted fragments are not selected by the A^* algorithm.....	27
Fig. 3-11	Given fragments distributed over the edit graph where the dotted fragments are not selected by the A^* algorithm.....	27
Fig. 3-12	The shortest path of Fig. 3-8.....	29
Fig. 3-13	The expanding tree of Fig. 3-8.....	29
Fig. 3-14	An example of two strings $X=acagcaacct$ and $Y=caagtccatccg$ and a fragment set M that $ M $ is 7: (a) The edit graph of X, Y and M . (b) The found shortest path of X, Y and M that the number of computed fragments is 4.....	30
Fig. 4-1	Comparing the number of computed fragments by using the A^* algorithm and the number of computed fragments by using the dynamic programming approach.....	33

List of Tables

Table 4-1 Experimental results.....	33
-------------------------------------	----

Chapter 1 Introduction

In computational biology, there are many important problems such as sequence alignment problem [NW70][SW81][ZBM98], multiple sequence alignment problem [CL88][BLP97], longest common subsequence (LCS) problem [H75][HS77][CR2002] and string matching problem [BM77][KMP77][ALP2004] etc.

In this thesis, we discuss a variation of the LCS problem. First, let us introduce the LCS problem defined as follows.

Def. 1-1

Longest common subsequence problem

Given two strings $X=x_1x_2\dots x_m$ and $Y=y_1y_2\dots y_n$, find a common subsequence $L=l_1l_2\dots l_p$ of X and Y such that p is maximized.

The LCS of two strings X and Y is denoted $\text{LCS}(X, Y)$. Let $|\text{LCS}(X, Y)|$ be the length of $\text{LCS}(X, Y)$. For instance, given $X=\text{agcgtta}$ and $Y=\text{gatgagt}$, $\text{LCS}(X, Y)$ is “aggt” and $|\text{LCS}(X, Y)|=4$. Up to now, many algorithms [H75][HS77] have been proposed to solve the problem. [H75] proposed a divide-and-conquer algorithm to solve the problem in linear space. [HS77] proposed a dynamic programming approach to solve the problem, and it introduced the sparse computing concept into the dynamic programming method.

We now introduce the LCS from fragments problem. First, given two strings X and

Y , a fragment of X and Y is a common substring of X and Y . Given two strings X and Y and a set M of fragments $\{f_1, f_2, \dots, f_h\}$ of X and Y , a common subsequence CS from M is a common subsequence between X and Y such that CS contains only some f_i 's or substrings of f_i 's. An LCS of X and Y from M is one of the common subsequence of X and Y from M which is the longest.

The LCS from fragments problem is defined as follows:

Def. 1-2

Longest common subsequence from fragments problem

Given two strings X and Y and a set M of fragments of X and Y , find a longest common subsequence of X and Y from M .

The LCS from fragments problem was first proposed in [BG2002], and a dynamic programming approach was proposed to solve the problem. In the dynamic programming approach, it has to compute all fragments in finding the solution. But, not all fragments appear in the found solution. Due to the reason, a new method is proposed in this thesis and the method can ignore the fragments which don't appear in solutions.

In the new method proposed in the paper, the concept of the A^* algorithm [LCTT91] is used. The method can help us to ignore the fragments which will not appear in solutions, and it can efficiently find a solution for the LCS from fragments problem. Next, let us introduce the score measure used in this thesis.

1.1 The Edit Distance

Given two strings X and Y , the edit distance between X and Y is the minimum number of deletions, substitutions and insertions to transform X to Y . For example, let $X=\text{aat-ct}$ and $Y=\text{aatgag}$. We may align the strings as follows:

$$X = \text{a a t - - c t}$$
$$Y = \text{a a t g a - g}$$

As can be seen, X can be transformed to Y by executing two insertions, one deletion and one substitution. It can also be proved that the edit distance between X and Y is $2+1+1=4$.

It is well known that the edit distance finding problem is equivalent to the longest common subsequence finding problem.

1.2 The Edit Graph Shortest Path Problem

In this section, we are interested in the longest common subsequence from fragments problem. In this section, we shall show that this problem can be viewed as a graph searching problem. The graph is defined an edit graph and the problem is to find a shortest path from a certain starting node to a certain terminal node.

Let us illustrate the edit graph through an example. Suppose we are given $X=acatat$ and $Y=atacat$. Then, the initial edit graph is showed in Fig. 1-1. In the edit graph, for a node p , let $x(p)$ and $y(p)$ to be the x -coordinate and y -coordinate of p , respectively. For instance, for node $(2, 3)$, $x((2, 3))=2$ and $y((2, 3))=3$. For node $(4, 5)$, $x((4, 5))=4$ and $y((4, 5))=5$.

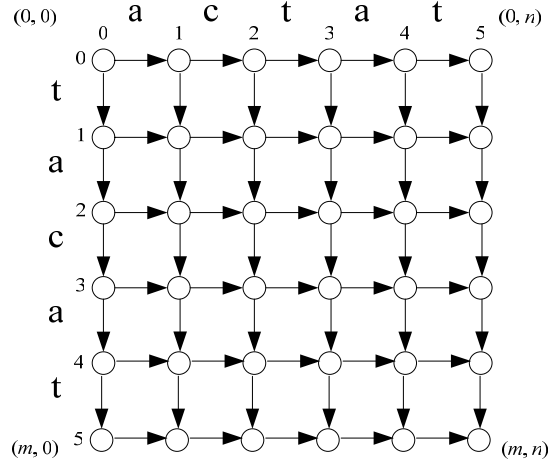


Fig. 1-1 The edit graph of two strings $X=tacat$ and $Y=actat$.

Suppose we further have fragments ta , ac and at . Then we denote these fragments as diagonal paths as shown in Fig. 1-2. We denote a path by (i, j, k) where i, j and k stand for the starting positions in X and Y and the length of the matching substring, respectively. For instance, given two strings $X=tacat$ and $Y=actat$ and $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$, $f_{(2, 1, 2)}$, $f_{(1, 3, 2)}$ and $f_{(4, 4, 2)}$ represent the three common substrings ac , ta and at , of X and Y , respectively.

In addition, for a fragment f , the node $(i-1, j-1)$ of f is denoted as $start(f)$ and the node $(i+k-1, j+k-1)$ of f is denoted by $end(f)$. For example, consider Fig. 1-2. $start(f_{(2, 1, 2)})$ is

the node $(1, 0)$ and $start(f_{(4, 4, 2)})$ is the node $(3, 3)$. $end(f_{(2, 1, 2)})$ is the node $(3, 2)$ and $start(f_{(4, 4, 2)})$ is the node $(5, 5)$.

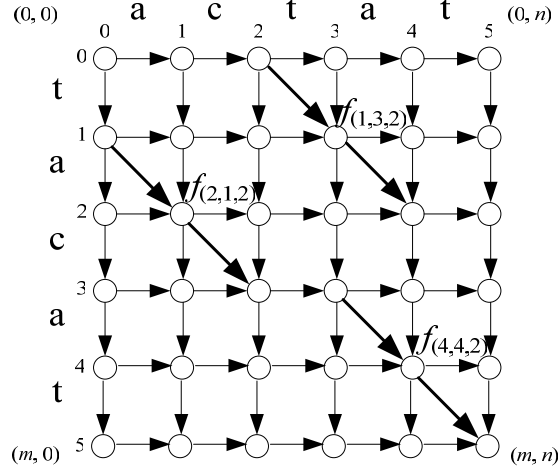


Fig. 1-2 The edit graph of two strings $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$.

For each horizontal and vertical edge, we associate it with cost 1 and for each diagonal edge, we associate it with cost 0. Our longest common subsequence from fragments problem can now be viewed as a shortest path finding problem where the path is from $(0, 0)$ to (m, n) . Fig. 1-3 shows the shortest path of Fig. 1-2.

After obtaining the shortest path, we obtain the solution by only retaining the substring in the fragments. There for the case shown above, the found solution is *acat*.

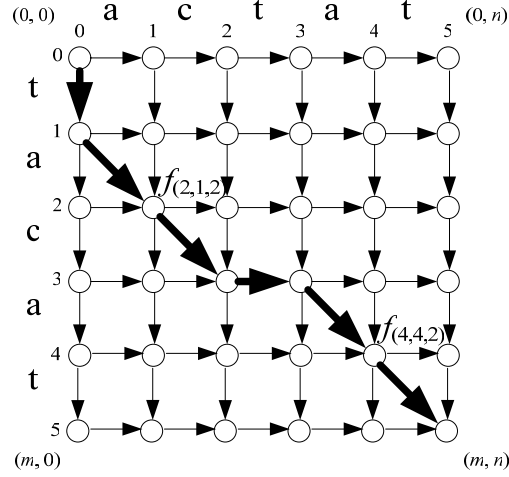


Fig. 1-3 The shortest path of two strings $X=tacat$ and $Y=actat$ and the set $M=\{f_{(2,1,2)}, f_{(1,3,2)}, f_{(4,4,2)}\}$.

Now, let us consider a situation. Would all fragments appear in the found shortest path? The answer is “Not necessarily”. For example, consider Fig. 1-2 and Fig. 1-3, fragment $f_{(1,3,2)}$ doesn’t appear in the found shortest path of X and Y and the set M . Thus we proposed a new method using A^* algorithm to solve the problem. Through this method, we can filter out the fragments which wouldn’t appear in the solution, and it can efficiently find the solution.

In the following sections, the dynamic programming approach proposed by Baker and Giancarlo will be introduced in Chapter 2. The A^* algorithm used to solve the LCS from fragments problem will be discussed in Chapter 3. Experimental results will be presented in Chapter 4. Finally, in Chapter 5, concluding statements will be presented.

Chapter 2 A Dynamic Programming Approach

In this chapter, we introduce a well-known dynamic programming approach proposed by Baker [BG2002], to solve the LCS from fragments problem. In section 2.1, the approach will be introduced. An example will be discussed in section 2.2. Finally, an application of the approach will be presented in section 2.3.

2.1 Baker-Giancarlo Approach

As we discussed in Chapter 1, the dynamic programming approach by Baker can be considered to solve the shortest path finding problem. The problem is equivalent to the shortest path finding problem. That is the LCS from fragments problem is to find a shortest path in an edit graph $[0m, 0n]$ from $(0, 0)$ to (m, n) . The edit distance is used to be the cost measure. In the following, a simple example is given to describe the main idea of the dynamic programming approach.

Given two strings $X=tacat$ and $Y=actat$ and a fragment set $M=\{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$, the edit graph is as shown in Fig. 2-1(a). The question is how to find a shortest path from $(0, 0)$ to (m, n) by the aid of these fragments. The costs of these fragments are hence examined by x -coordinate increasing order one by one. Among the three fragments, $f_{(1, 3, 2)}$ is first processed because $x(start(f_{(1, 3, 2)}))$ is the lowest. There are two horizontal edges

between $(0, 0)$ and $f_{(1, 3, 2)}$, therefore, the cost of $f_{(1, 3, 2)}$ is 2. Likewise, the cost of $f_{(2, 1, 2)}$ is 1. Finally, we deal with $f_{(4, 4, 2)}$. The cost of $f_{(4, 4, 2)}$ seems to be 6(3 horizontal edges and 3 vertical edges). However, we can reduce this cost by connecting $f_{(4, 4, 2)}$ to $f_{(2, 1, 2)}$. The new cost will be the cost of $f_{(2, 1, 2)}$ plus 1(one horizontal edge, a bridge for $f_{(2, 1, 2)}$ and $f_{(4, 4, 2)}$), and it is equal to 2. Since $f_{(4, 4, 2)}$ has reached the destination node (m, n) , we can concatenate $f_{(2, 1, 2)}$ and $f_{(4, 4, 2)}$ to form the final solution as shown in Fig. 2-1(b). From the above example, we know the key point of this approach is how to select and combine proper fragments, and we will discuss it in detail in the following.

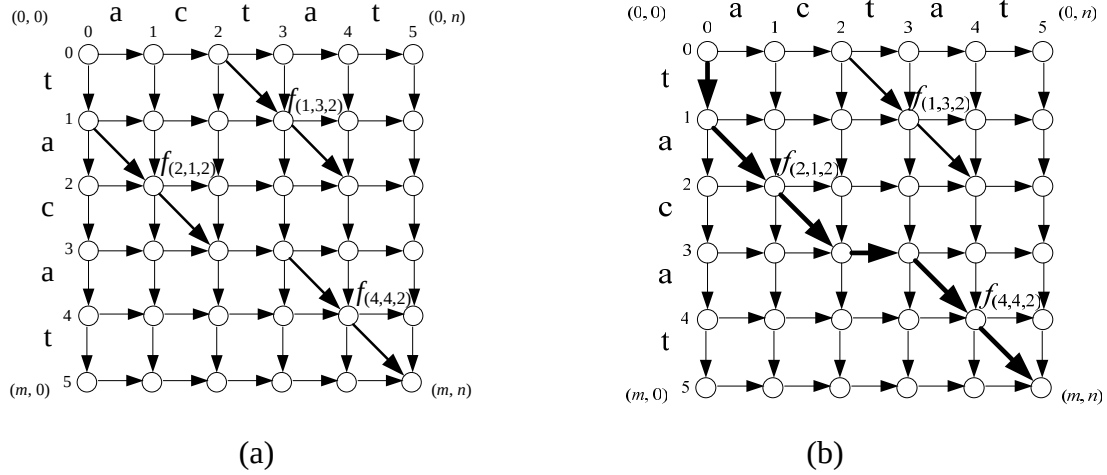


Fig. 2-1 An example of $X = \text{tacat}$ and $Y = \text{actat}$ and a set $M = \{f_{(2, 1, 2)}, f_{(1, 3, 2)}, f_{(4, 4, 2)}\}$.

(a) The edit graph of X and Y and the set M . (b) The shortest path of X and Y and the set M .

In the approach, three relations, left relation, above relation and precedence relation, among fragments are used, and the definitions of them are as follows.

Def. 2-1

Left Relation

A fragment f' is left of fragment f , if some point of f' besides $start(f')$ is to the left of $start(f)$ on a horizontal line through $start(f)$, or $start(f)$ lies on f' and $x(start(f')) < x(start(f))$.

Def. 2-2

Above Relation

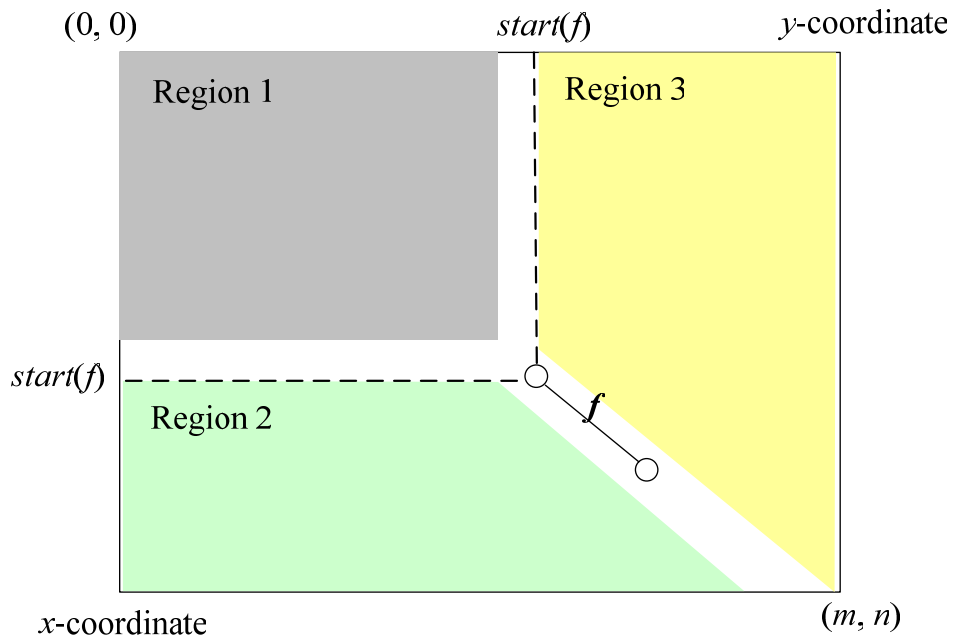
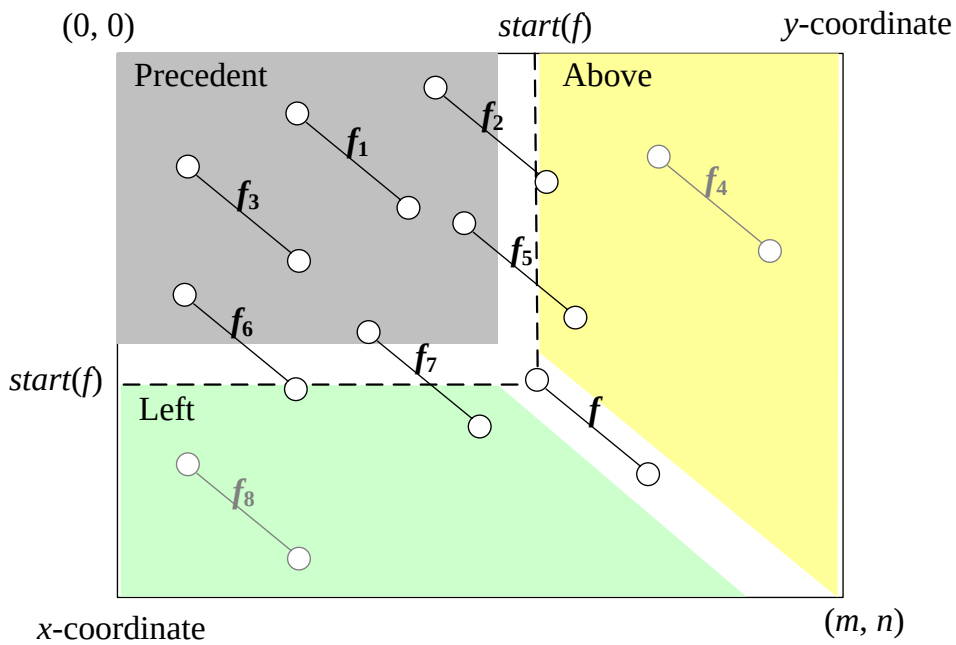
A fragment f' is above fragment f , if some point of f' besides $start(f')$ is strictly above $start(f)$ on a vertical line through $start(f)$.

Def. 2-3

Precedent Relation

Fragment f' precedes fragment f , if $x(end(f')) < x(start(f))$ and $y(end(f')) < y(start(f))$.

Consider Fig. 2-2. Among two fragment f and f' , if $start(f')$ is located in region 1, the relation of two fragments f and f' can be easily decided by $end(f')$. If $end(f')$ falls on region 1, f' precedes f . Furthermore, if $end(f')$ falls on region 2(3), f' is left(above) of f .


 Fig. 2-2 Three regions of fragment f .

 Fig. 2-3 The relations between fragments $f_1 \sim f_8$ and f' .

Take Fig. 2-3 as an example. f_1 and f_3 have precedence relations with f , f_6 and f_7 have left relations with f , and f_2 and f_5 have above relations with f . Furthermore, f_4 and f_8 don't have relationship with f . Moreover, we can reduce the cost of f by connecting the other fragments having relations with f . For left relations, f_6 and f_7 , f can connect f_7 due to the fact that f_7 is nearer than f_6 for f . Similarly, for the above relations, f_2 and f_5 , f should connect f_5 .

For a fragment f , the nearest left fragment and the nearest above fragment are denoted by symbols $visl(f)$ and $visa(f)$, respectively. The formal definitions are as follows.

Def. 2-4

$visl(f)$

The first fragment to the left of $start(f)$ if such exists, and undefined otherwise.

Def. 2-5

$visa(f)$

The first fragment above $start(f)$ if such exists, and undefined otherwise.

In Fig. 2-3, $visl(f)$ is f_7 and $visa(f)$ is f_5 . As to which you may ask which fragment preceding f is to be used. In fact, we can't give an answer right away, since the selection of one precedence fragment to connect must consider another factor. We must consider global cost. The one with the lowest cost (its own cost + the distance to f) will be selected.

The cost of each fragment is to be computed by row major recursively. Hence, all fragments are needed to be sorted by lexicographic sort. In preprocessing, lexicographic sort is used. The main algorithm contains two phases, one is to compute $visl(f)$ and $visa(f)$, and then use these information to compute the cost for every fragment. We give an example in the following section.

2.2 An Example

In this section, an example is used to explain the main algorithm which contains two phases. Given two strings $X=atcggatcgc$ and $Y=ccgtccgac$ and a set $M=\{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}$ as shown in Fig. 2-4. In the pre-processing, M is sorted by lexicographic sort and the sorted $M=\{f_{(2, 4, 2)}, f_{(3, 2, 2)}, f_{(3, 6, 2)}, f_{(5, 7, 2)}, f_{(7, 4, 2)}, f_{(8, 2, 2)}, f_{(8, 6, 2)}\}$.

In the first phase of the algorithm, $visl(f_{(3, 6, 2)})=f_{(2, 4, 2)}$, $visl(f_{(5, 7, 2)})=f_{(3, 2, 2)}$, $visa(f_{(5, 7, 2)})=f_{(3, 6, 2)}$, $visa(f_{(7, 4, 2)})=f_{(3, 2, 2)}$, $visl(f_{(8, 6, 2)})=f_{(7, 4, 2)}$ and $visa(f_{(8, 7, 2)})=f_{(2, 4, 2)}$ are found step by step by the sweep-line method.

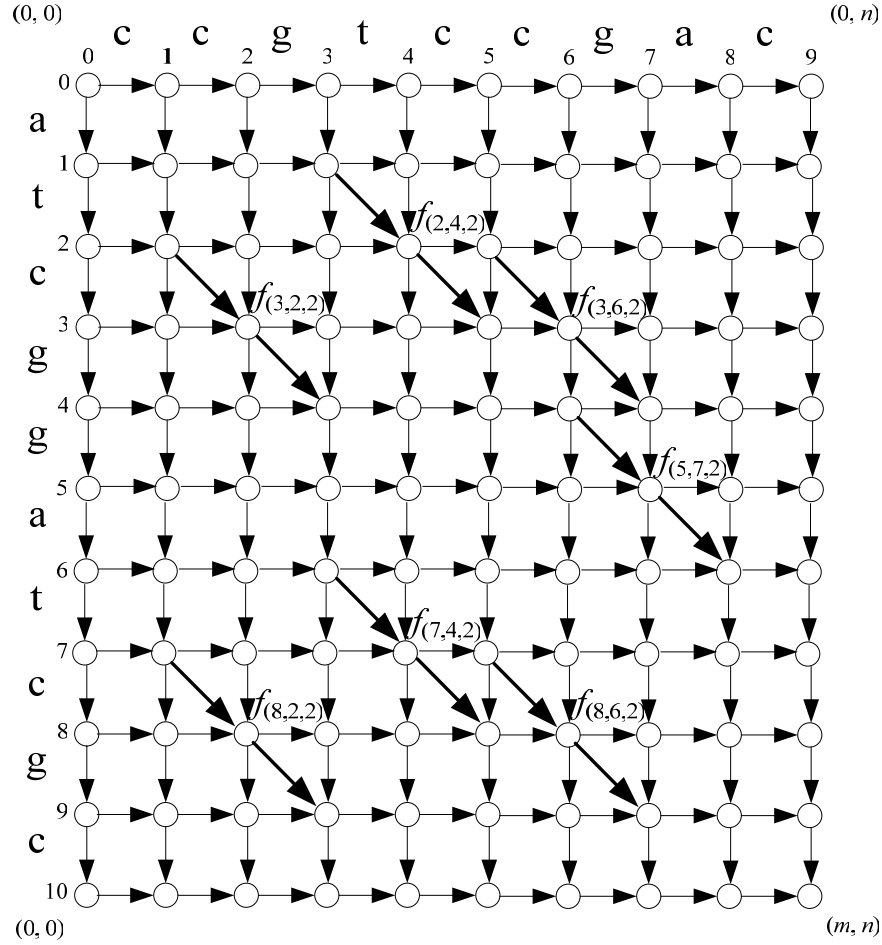


Fig. 2-4 The edit graph of two strings $X=atcggatcgc$ and $Y=ccgtccgac$ and a set $M=\{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}$

In the second phase of the algorithm, the result of computing the cost for each fragment f and the cost of node (m, n) is as shown in Fig. 2-5. Let us see how to compute these values and get a shortest path. The cost of $f_{(2, 4, 2)}$ is 4, because there are three horizontal edges and one vertical edge in the range from $(0, 0)$ to $f_{(2, 4, 2)}$. For $f_{(3, 2, 2)}$, because there are 1 horizontal edge and 2 vertical edges, the cost of $f_{(3, 2, 2)}$ is 3. Because $visl(f_{(3, 6, 2)})$ is $f_{(2, 4, 2)}$, the path from $(0, 0)$ to $f_{(3, 6, 2)}$ can cross $f_{(2, 4, 2)}$ due to getting a smaller cost, 5. In this case, $f_{(5, 7, 2)}$ can connect $f_{(3, 6, 2)}$, $f_{(3, 2, 2)}$ or $f_{(2, 4, 2)}$ to get the same result.

After computing all the cost for each fragment, because we want to find a shortest path from $(0, 0)$ to (m, n) , node (m, n) is needed to be computed by the same way. Thus, node (m, n) connects $f_{(8, 6, 2)}$ to get the smaller cost and the cost of node (m, n) is 9 (seeing Fig. 2-5). Then, the shortest path from $(0, 0)$ to (m, n) can be gotten as shown in Fig. 2-6, and it corresponds to “cgtcgc” which is exactly the LCS from fragments.

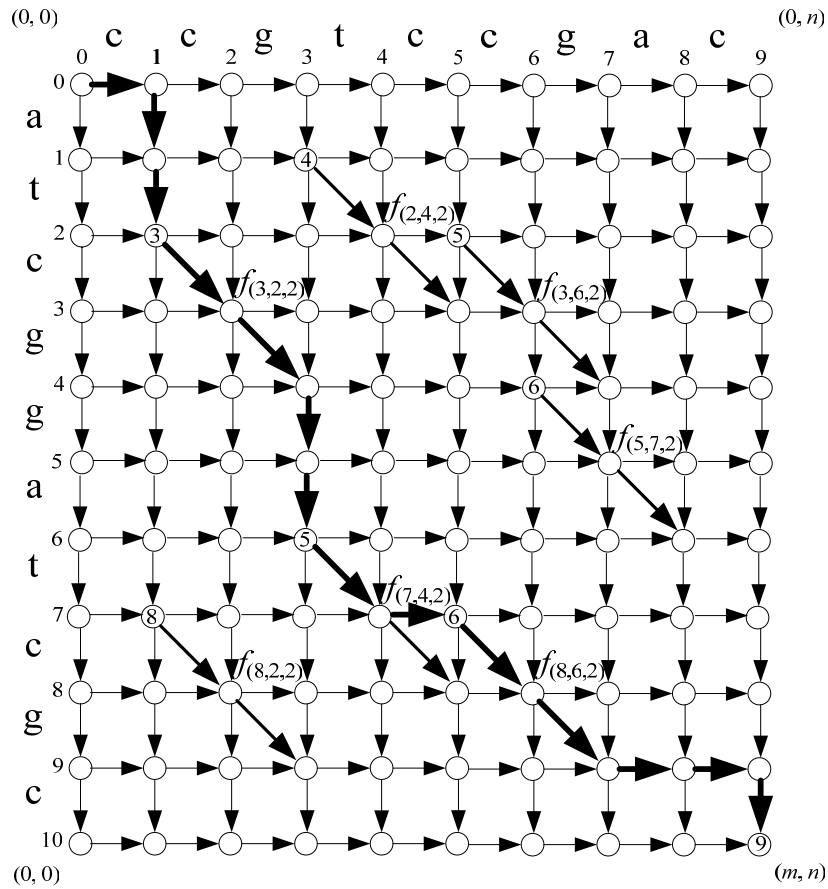


Fig. 2-6 The shortest path of two strings $X=atcggatcgc$ and $Y=ccgtccgac$ and the

$$\text{set } M = \{f_{(3, 2, 2)}, f_{(8, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(3, 6, 2)}, f_{(8, 6, 2)}, f_{(5, 7, 2)}\}.$$

Until now, the dynamic programming approach proposed by Baker and Giancarlo has

been discussed. In the following chapter, the A^* algorithm used to solve the LCS from fragments problem will be introduced. Of course, the algorithm also can be viewed to solve the shortest path finding problem.

Chapter 3 The A* Algorithm

In this chapter, we shall introduce the A^* algorithm for the shortest path finding problem. The edit distance is used to be the cost measure through this chapter. In the edit graph, for each horizontal and vertical edge, we associate it with cost 1 and for each diagonal edge, and we associate it with cost 0. In the following, we shall use one simple example to informally illustrate the basic idea of the A^* algorithm.

Consider Fig. 3-1. It's the edit graph of two strings $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2,1,2)}, f_{(1,3,2)}, f_{(4,4,2)}\}$. In the execution of the A^* algorithm, there are two sets, *found* and *unfound*. At the beginning, the *found* is null and *unfound* stores the two nodes $(0, 0)$ and (m, n) and the three given fragments, $f_{(2,1,2)}$, $f_{(1,3,2)}$ and $f_{(4,4,2)}$.

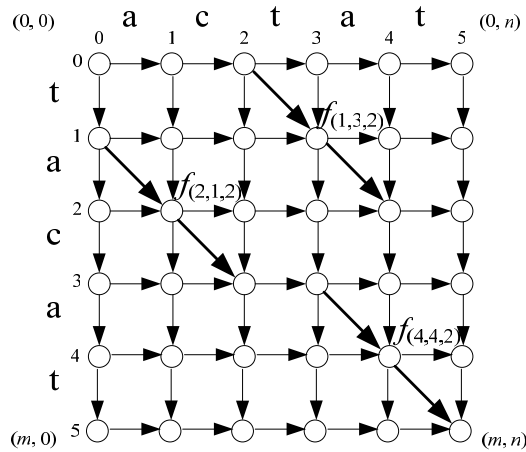


Fig. 3-1 The edit graph of two strings $X=tacat$ and $Y=actat$ and a set $M=\{f_{(2,1,2)}, f_{(1,3,2)}, f_{(4,4,2)}\}$.

In the first step, node $(0, 0)$ is selected from *unfound* to *found*, because there is no cost between it and the starting position of the shortest path from $(0, 0)$ to (m, n) . Then, expand the selected node $(0, 0)$ to the elements in *unfound* as shown in Fig. 3-2, and the costs of the paths from $(0, 0)$ to all elements in *unfound* are obtained. For example, the cost of the path from $(0, 0)$ to $f_{(2,1,2)}$ is 1, the cost of the path from $(0, 0)$ to $f_{(1,3,2)}$ is 2, the cost of the path from $(0, 0)$ to $f_{(4,4,2)}$ is 6, and the cost of the path from $(0, 0)$ to (m, n) is 10.

By the A* algorithm, for each element in *unfound*, the cost of the path from it to the ending node (m, n) is estimated. The details of the estimating method will be discussed later. Since the cost of the paths from each element in *unfound* to the ending node (m, n) can be estimated, the cost of the path from $(0, 0)$ to (m, n) through each element in *unfound* can be gotten. Consider Fig. 3-2. The cost of the path from $(0, 0)$ to (m, n) through $f_{(2,1,2)}$ is 2, the cost of the path from $(0, 0)$ to (m, n) through $f_{(1,3,2)}$ is 5, the cost of the path from $(0, 0)$ to (m, n) through $f_{(4,4,2)}$ is 6 and the cost of the path from $(0, 0)$ to (m, n) is 10.

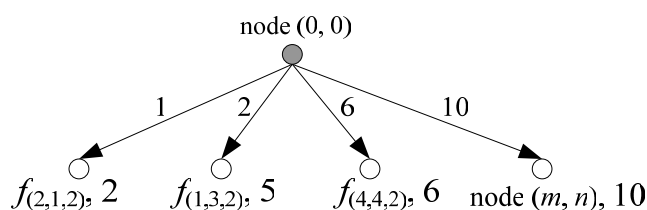


Fig. 3-2 Node $(0, 0)$ selected to expand of Fig. 3-1.

Next, in *unfound*, the element whose cost of the path from $(0, 0)$ to (m, n) through it is the lowest, is selected to be the expanding element. In our case, $f_{(2, 1, 2)}$ is selected from *unfound* to *found*, and $f_{(2, 1, 2)}$ is expanded to the elements in *unfound* as shown in Fig. 3-3. Thus the cost of the path from $(0, 0)$ to $f_{(1, 3, 2)}$ through $f_{(2, 1, 2)}$ is $1+3=4$, the cost of the path from $(0, 0)$ to $f_{(4, 4, 2)}$ through $f_{(2, 1, 2)}$ is $1+1=2$ and the cost of the path from $(0, 0)$ to (m, n) through $f_{(2, 1, 2)}$ is $1+5=6$. As well, from each element in *unfound*, the cost of the paths from it to (m, n) can be estimated. Through this estimation, we know that the path from $(0, 0)$ to (m, n) through $f_{(2, 1, 2)}$ and $f_{(1, 3, 2)}$ is with cost 5, the path from $(0, 0)$ to (m, n) through $f_{(2, 1, 2)}$ and $f_{(4, 4, 2)}$ is with cost 2 and the path from $(0, 0)$ to (m, n) through $f_{(2, 1, 2)}$ is with cost 6.

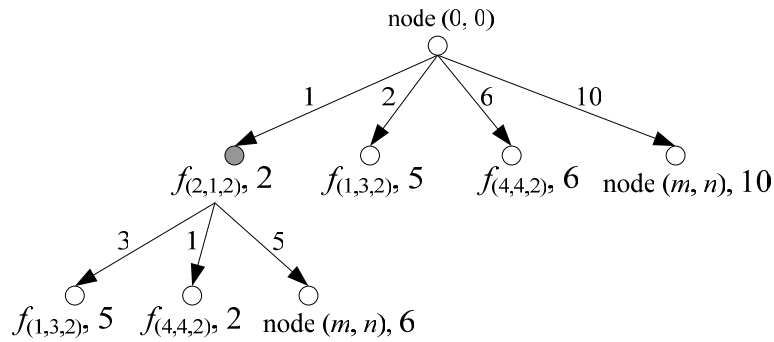
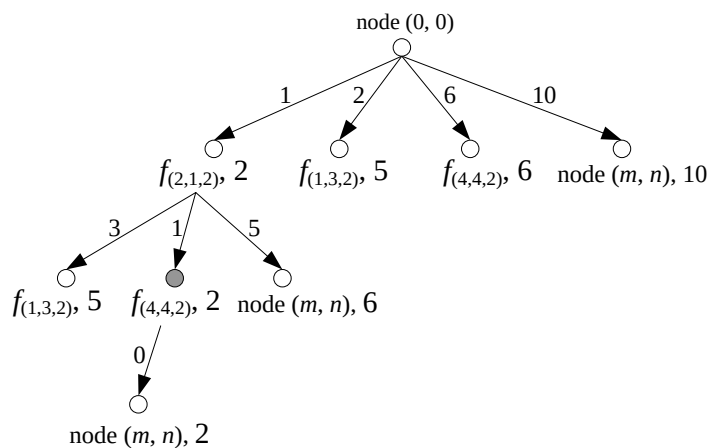
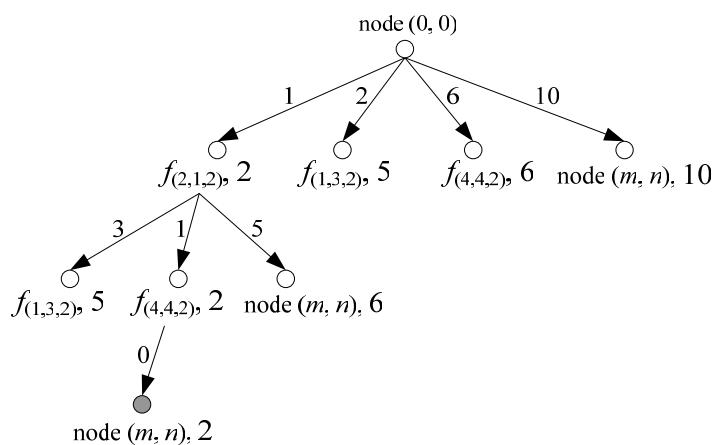


Fig. 3-3 $f_{(2, 1, 2)}$ selected to expand of Fig. 3-1.

In the same way, as shown in Fig. 3-4, $f_{(4, 4, 2)}$ is selected from *unfound* to *found* because the cost of the path from $(0, 0)$ to (m, n) through $f_{(2, 1, 2)}$ is lowest among the elements in *unfound*. We expand it to the elements in *unfound*.

Fig. 3-4 $f_{(4,4,2)}$ selected to expand of Fig. 3-1.

As shown in Fig. 3-5. Node (m, n) is selected next. Because the selected element is node (m, n) , the shortest path from $(0, 0)$ to (m, n) is found by A* algorithm. Fig. 3-5 shows that the found shortest path is from $(0, 0)$ to (m, n) through two fragments $f_{(2,1,2)}$ and $f_{(4,4,2)}$, and the path corresponds the shortest path of the edit graph of X , Y and M as shown in Fig. 3-6.

Fig. 3-5 Node (m, n) selected to expand of Fig. 3-1.

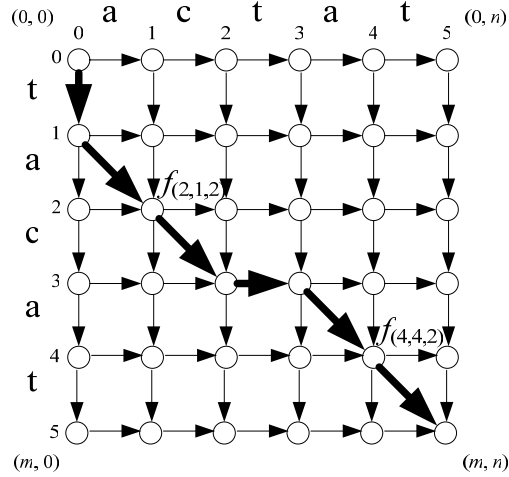


Fig. 3-6 The shortest path of two strings $X=tacat$ and $Y=actat$ and $M=\{f_{(2,1,2)}, f_{(1,3,2)}, f_{(4,4,2)}\}$.

As shown in Fig. 3-6. The found shortest path doesn't contain the fragment $f_{(1,3,2)}$. The reader can see that $f_{(1,3,2)}$ was not contained by the A^* algorithm. This is the advantage of our A^* algorithm. Many fragments may not need to be considered. The details of the A^* algorithm to solve the shortest path finding problem will be discussed in the below.

Consider the partially expanded tree shown in Fig. 3-7. Let p denote a node. Let $Dist(p)$ denote the length of the shortest path from the root of the tree to a terminal node through p . Let $D(p)$ denote the length of the shortest path from the root of the tree to p . Let $E(p)$ denote the length of the shortest path from p to a terminal node. Then $Dist(p)=D(p)+E(p)$.

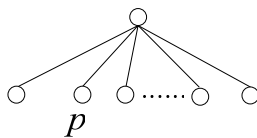


Fig. 3-7 p is a node of the expanded tree.

For A* algorithm, there is an estimation scheme which estimates the lower bound of $E(p)$. Let $E^*(p)$ denote the estimated $E(p)$. Then $Dist^*(p) = D(p) + E^*(p)$. The A* algorithm always selects the node with the lowest $Dist^*(p)$ to expand and stops if the selected node is a terminal node.

To apply the A* algorithm to our problem, we use the edit graph as the input data. We always expand the nodes related to the fragments. For instance, consider Fig. 3-8. The nodes to be expanded at the first level are points $(3, 2)$, $(2, 4)$, $(8, 2) \dots (5, 7)$.

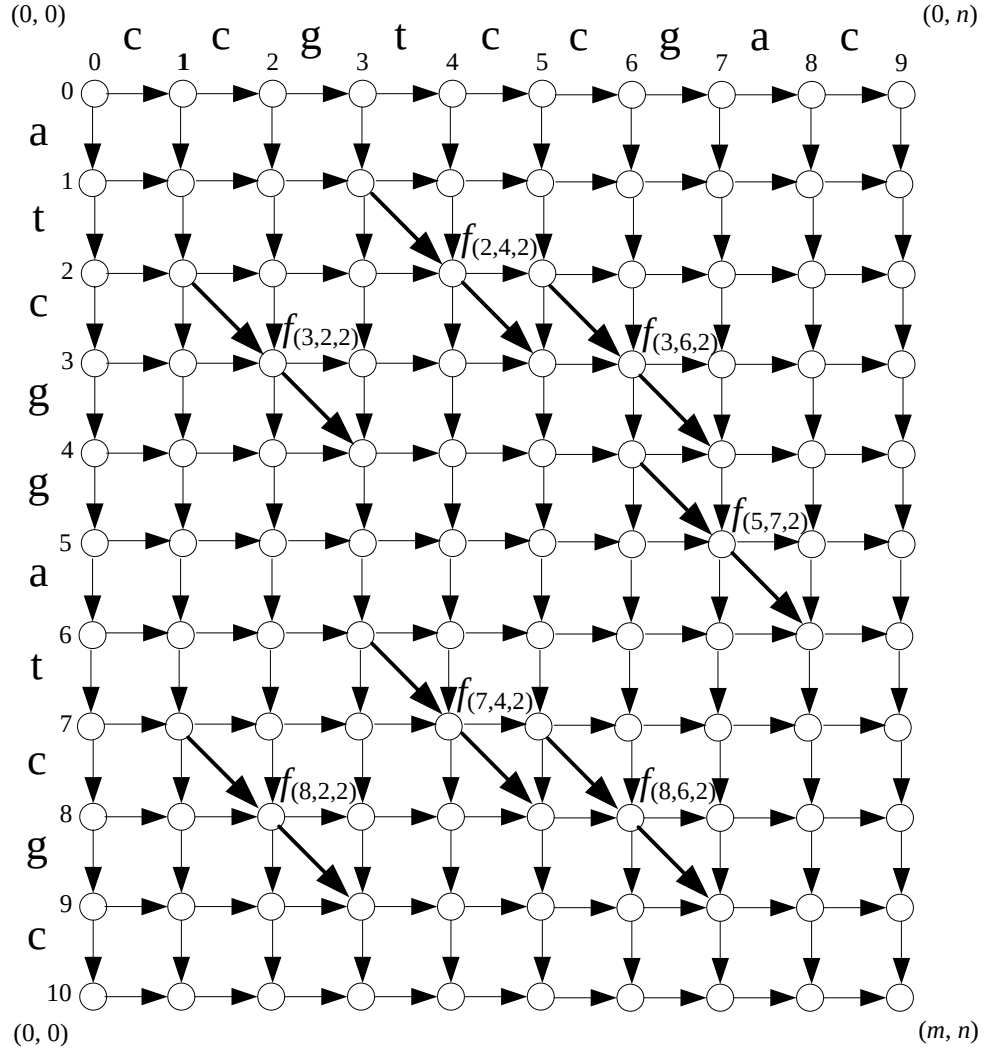


Fig. 3-8 The edit graph of two strings $X=atcggatcgc$ and $Y=ccgtccgac$ and a set $M=\{f_{(2,4,2)}, f_{(3,2,2)}, f_{(3,6,2)}, f_{(5,7,2)}, f_{(7,4,2)}, f_{(8,2,2)}, f_{(8,6,2)}\}$.

Now, for each node p , we define $E^*(p)$ as follows. Let $S_{row}(p)$ denote the number of the rows in the area from p to (m, n) which do not contain diagonal edges and the number of the columns in the area from p to (m, n) which do not contain diagonal edges is denoted by $S_{col}(p)$. For instance, in Fig. 3-8, $S_{row}((0, 0))=2$ and $S_{row}((4, 3))=1$. $S_{col}((0, 0))=2$ and $S_{col}((4, 3))=1$. Obviously, from a node p to (m, n) of the edit graph, $E^*(p)$ can be defined as follows: $E^*(p) = S_{row}(p) + S_{col}(p)$.

In addition, we may use some other information to cut down the searching space. In Fig. 3-8, consider the fragment $f_{(3, 2, 2)}$. Any solution consisting of this fragment will not contain use the fragment $f_{(2, 4, 2)}$. Therefore, when we consider $f_{(3, 2, 2)}$, we may ignore $f_{(2, 4, 2)}$. We now define the domination relation as follows: For two fragments f and f' , f' is dominated by f , if $x(\text{end}(f')) > x(\text{end}(f))$ and $y(\text{end}(f')) > y(\text{end}(f))$. In our algorithm, suppose we are considering a fragment f , we shall ignore all of the fragments which are not dominated by f . For instance, in Fig. 3-8, $f_{(3, 2, 2)}$ dominates $f_{(5, 7, 2)}$, $f_{(7, 4, 2)}$ and $f_{(8, 6, 2)}$. $f_{(2, 4, 2)}$ dominates $f_{(3, 6, 2)}$, $f_{(8, 6, 2)}$ and $f_{(5, 7, 2)}$, and $f_{(7, 4, 2)}$ dominates $f_{(8, 6, 2)}$.

In addition, for two fragments f and f' , we define $Len(f, f')$ as the number of horizontal and vertical edges between f and f' . Let us consider Fig. 3-8. $Len(f_{(2, 4, 2)}, f_{(3, 6, 2)})$ is 1. $Len(f_{(2, 4, 2)}, f_{(5, 7, 2)})$ is 2. $Len(f_{(2, 4, 2)}, f_{(8, 6, 2)})$ is 4.

3.1 The Algorithm

The algorithm proposed in this thesis consists of two phases, preprocessing and main algorithm. In Step 3 of the main algorithm as below, $E^*(f)$ has to be used in for loop. For each fragment f , since $E^*(f)$ doesn't change, we only compute $E^*(f)$ once for each f . In the preprocessing, we compute $E^*(f)$ for each f . In the main algorithm, a value *Link* is prepared for each fragment f and it will be used in recovering the shortest path step.

Algorithm

Let *Par* be pointer.

Input: Two strings $X=x_1x_2\dots x_m$ and $Y=y_1y_2\dots y_n$, and a set M of fragments.

Output: A shortest path from $(0, 0)$ to (m, n)

Step 1. Consider two nodes $(0, 0)$ and (m, n) as fragments $f_{(1, 1, 0)}$ and $f_{(m, n, 0)}$, respectively.

Put fragments $f_{(1, 1, 0)}$ and $f_{(m, n, 0)}$ and set M into set *unfound*. Let set *found*= ϕ .

Step 2. For each fragment f in set *unfound* except fragment $f_{(1, 1, 0)}$, let $D(f)=\infty$ and

$Dist^*(f)=\infty$. For $f_{(1, 1, 0)}$, let $D(f_{(1, 1, 0)})=0$ and $Dist^*(f_{(1, 1, 0)})=E^*(f_{(1, 1, 0)})$.

Step 3. Do until $f = f_{(m, n, 0)}$

Select f from set *unfound* such that $Dist^*(f)$ is the smallest, and remove f from *unfound* to *found*.

For every f' in *unfound* which is dominated by f do

Let $\overline{D(f')} = D(f) + Len(f, f')$.

If $\overline{D(f')} < D(f')$, set $f'.Link = f$.

Let $D(f') = \min\{D(f'), \overline{D(f')}\}$.

Let $Dist^*(f') = D(f') + E^*(f')$.

End For

End Do

Step 4. Set $Par = f_{(m, n, 0)}$.

While *Par* is not *null*

Print (i, j) pair pointed to by *Par*

Advance *Par*

End While

Consider Fig. 3-9. It shows the diagonal line of edit graph. If the given fragments are distributed over the diagonal line of the edit graph, the A* algorithm can not filter out many fragments which will not appear in the solution. For instance, in Fig. 3-11, the fragments are distributed over the diagonal line of the edit graph. There are few dotted fragments filtered by the A* algorithm. Furthermore, if the given fragments are distributed over the edit graph, the A* algorithm can filter out many fragments which will not appear in the solution. Consider Fig. 3-11. The fragments are distributed over the edit graph, and there are many dotted fragments filtered by the A* algorithm. Hence, when given fragments are distributed over the edit graph, the performance of the A* algorithm is better.

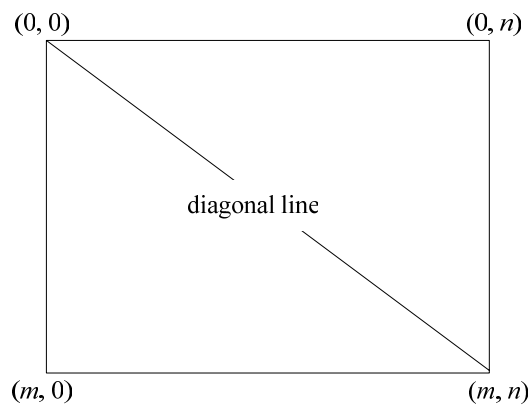


Fig. 3-9 The diagonal line of edit graph.

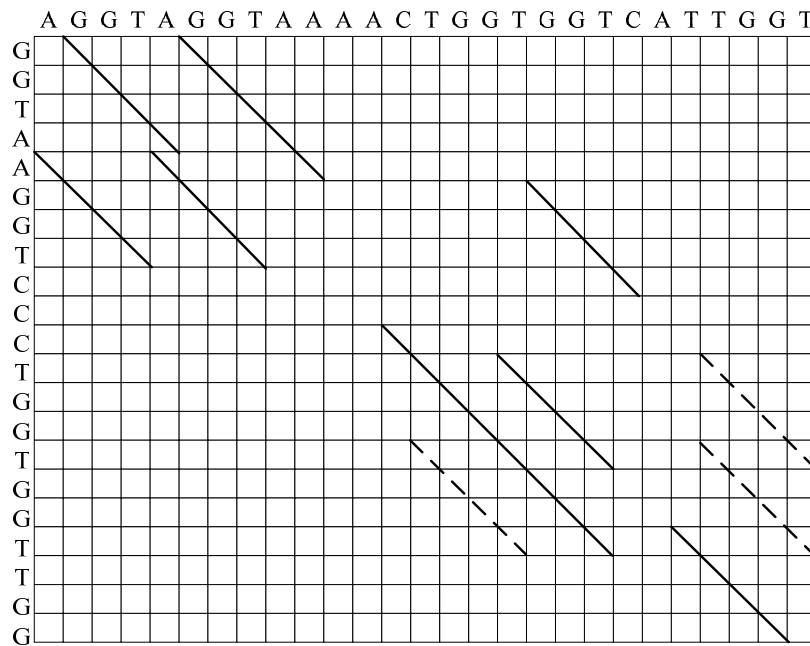


Fig. 3-10 Given fragments distributed over the diagonal line of the edit graph where the dotted fragments are not selected by the A* algorithm.

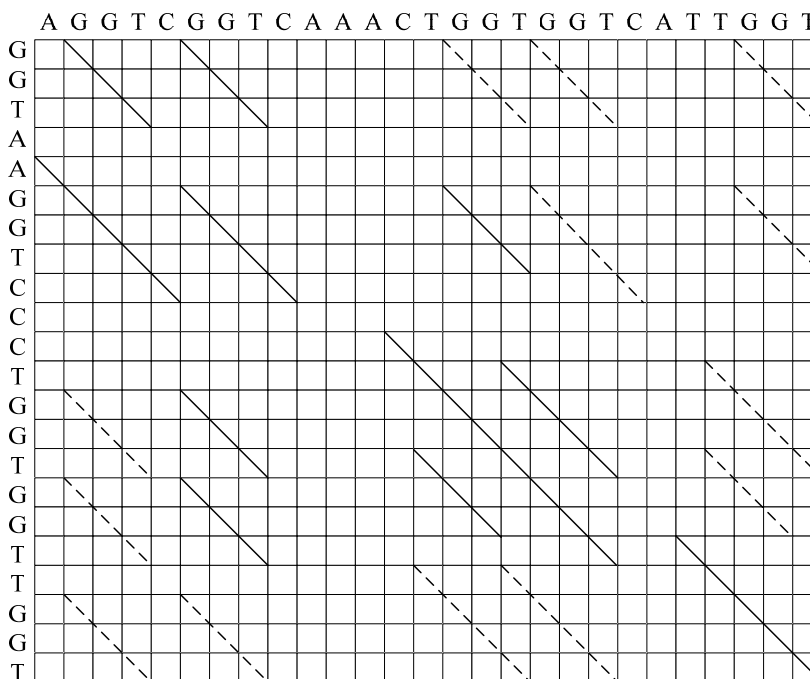


Fig. 3-11 Given fragments distributed over the edit graph where the dotted fragments are not selected by the A* algorithm.

After introducing the A* algorithm to the shortest path finding problem, let us see two examples to explain it again in next section.

3.2 Examples

In this section, we use two examples to show the results of applying the A* algorithm to solve the LCS from fragments problem. In the first example, two strings X and Y are “atcggatcgc” and “ccgtccgac”, respectively, and a fragment set $M = \{f_{(2, 4, 2)}, f_{(3, 2, 2)}, f_{(3, 6, 2)}, f_{(5, 7, 2)}, f_{(7, 4, 2)}, f_{(8, 2, 2)}, f_{(8, 6, 2)}\}$ as shown in Fig. 3-8. In the processing of A* algorithm, nodes $(0, 0)$ and (m, n) are considered as two fragments $f_{(1, 1, 0)}$ and $f_{(m, n, 0)}$, respectively. The order of selected fragments is $f_{(1, 1, 0)}, f_{(3, 2, 2)}, f_{(2, 4, 2)}, f_{(7, 4, 2)}, f_{(8, 6, 2)}$ and $f_{(m, n, 0)}$. The tree expanding of this example is as shown in Fig. 3-13. When node (m, n) is selected, the shortest path of X , Y and M is found successfully. The found shortest path from $(0, 0)$ to (m, n) of X and Y as shown in Fig. 3-12 corresponds to “cgtcg” which is the LCS from fragments of X , Y and M .

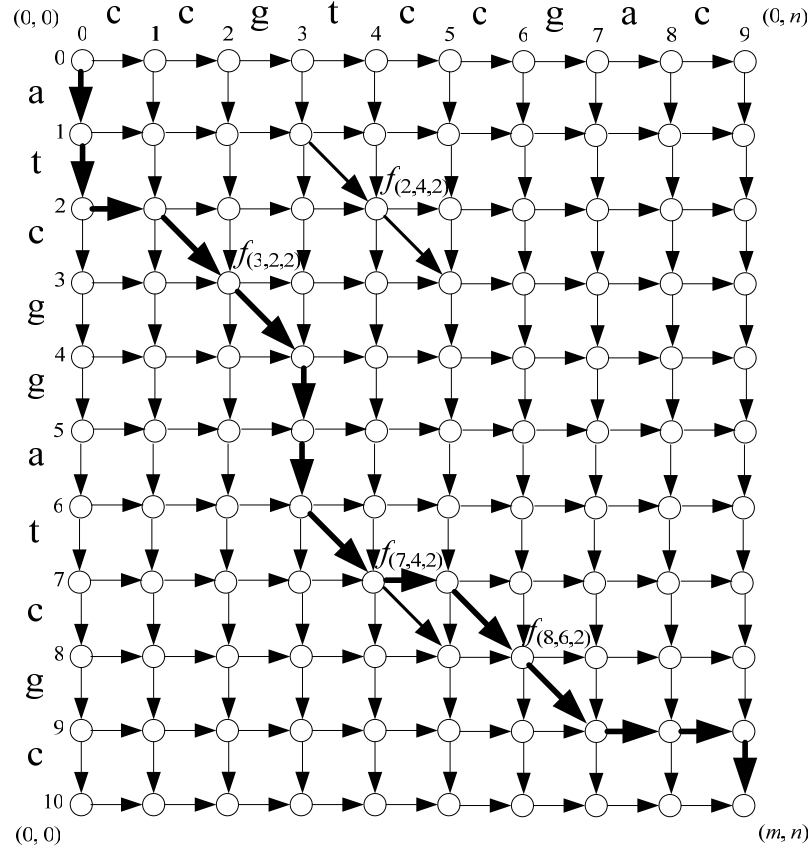


Fig. 3-12 The shortest path of Fig. 3-8.

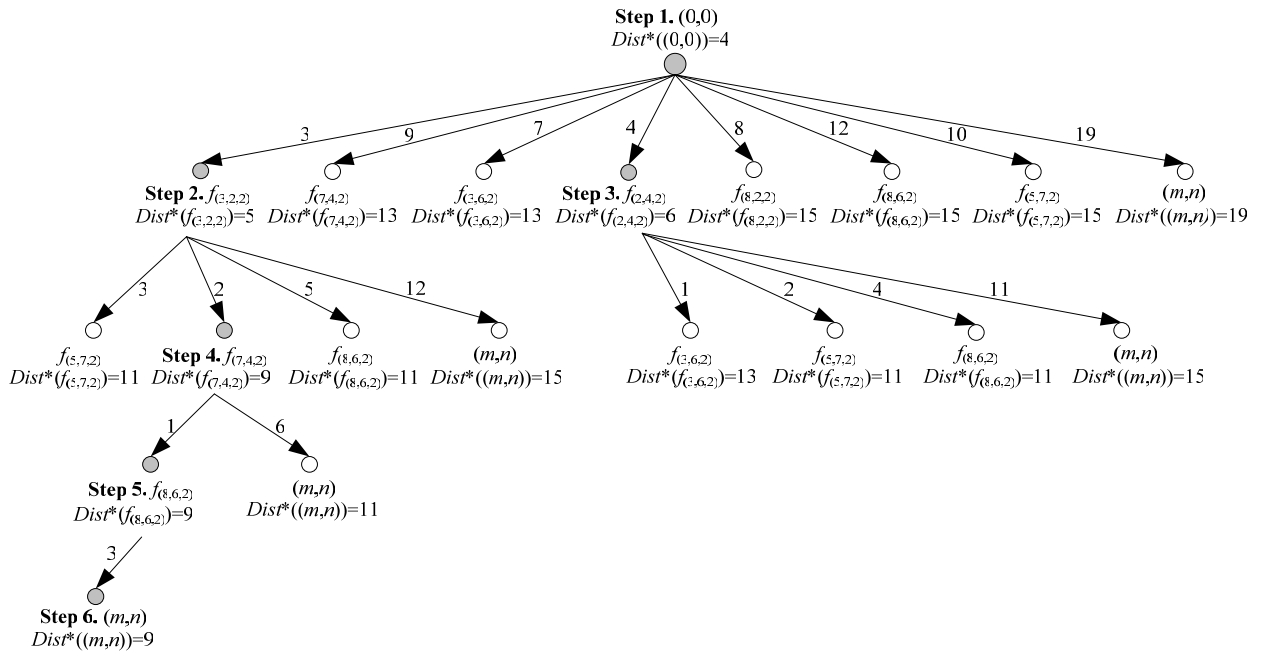
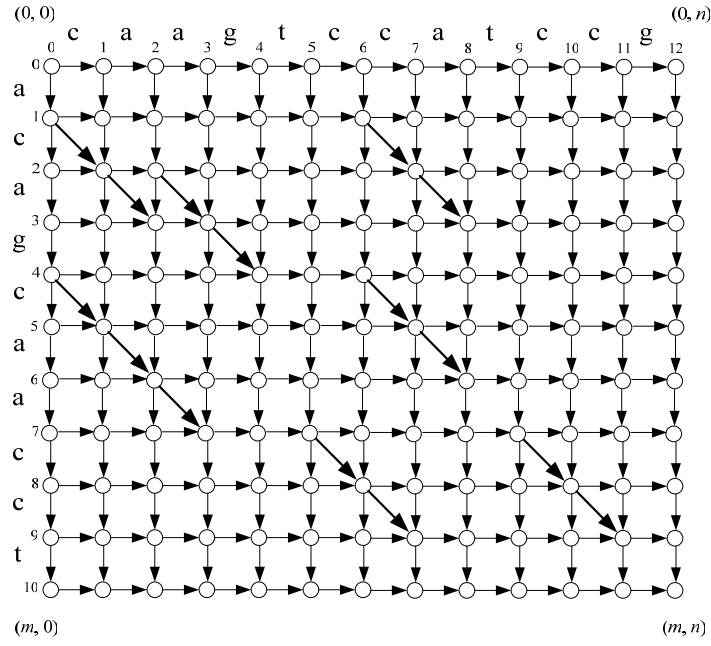
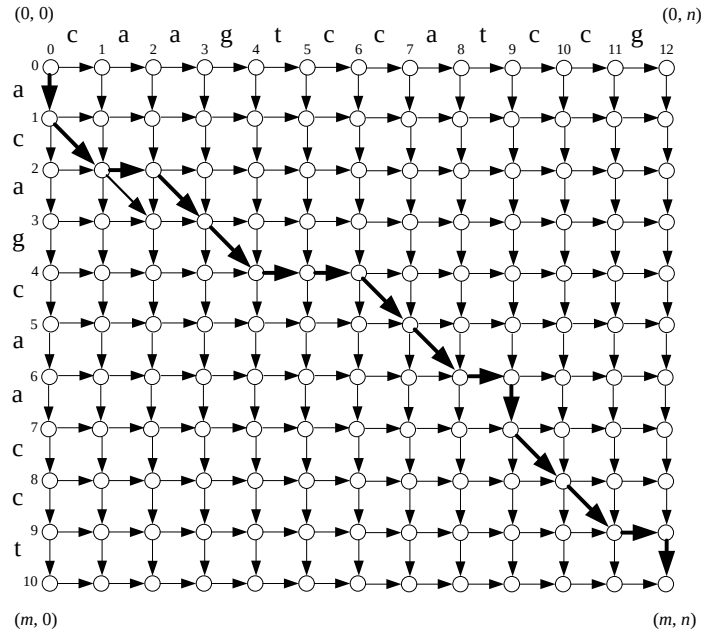


Fig. 3-13 The expanding tree of Fig. 3-8.



(a)



(b)

Fig. 3-14 An example of two strings $X=acagcaacct$ and $Y=caagtcacccg$ and a fragment set M that $|M|$ is 7: (a) The edit graph of X , Y and M . (b) The found shortest path of X , Y and M that the number of computed fragments is 4.

In the second example, strings X and Y are “acagcaacct” and “caagtccatccg”, respectively, and fragment set $M=\{f_{(2,1,2)}, f_{(2,7,2)}, f_{(3,3,2)}, f_{(5,1,3)}, f_{(5,7,2)}, f_{(8,6,2)}, f_{(8,10,2)}\}$ as shown in Fig. 3-14(a). In the processing of A* algorithm, four fragments, $f_{(2,1,2)}$, $f_{(3,3,2)}$, $f_{(5,7,2)}$ and $f_{(8,10,2)}$, are selected as shown in Fig. 3-14(b). By the path, the solution of LCS from fragments can be found successfully, and it is “cagcacc”.

In the above two examples, not all given fragments have to be computed in finding the shortest path. In the dynamic programming approach proposed by Baker and Giancarlo, in finding the shortest path, all fragments need to be computed. In general, the number of fragments needed to be computed by using the A* algorithm is less than that by using the dynamic programming approach. However, it is still possible to have to compute all fragments in the worst case in the A* algorithm.

Chapter 4 Experiments

In this section, we shall present some experimental results processed by the A^* algorithm proposed in this thesis and the dynamic programming approach proposed by Baker and Giancarlo. In the experiments, 6 Hepatitis B Virus and 10 gene strings were used. The 6 Hepatitis B Viruses were HT, A4, pAD14, HMA, 8884H and 7768H. The 10 gene strings were CYP3A26, LOC489851, At2g30770, At1g11610, Fmo3, FMO3, P0452F10.9, P0452F10.11, B1131G07.28-2, B1131G07.28-1, which are considered to belong to the a same group, CYP3A7, in NCBI Website. Table 4-1 shows the definition of terms used in the experiments. The experimental results are shown in Table 4-1. In Fig. 4-1, we compare the number of computed fragments by using the A^* algorithm and the number of computed fragments by using the dynamic programming approach.

S_1 : The name of the first string.

S_2 : The name of the second string.

Len_1 : The length of the first string.

Len_2 : The length of the second string.

T : The threshold of the length of fragments.

M : The number of fragments whose length are greater than and equal to the threshold T .

A^* : The number of fragments which need to be computed by the A^* algorithm.

DP : The number of fragments which need to be computed by the dynamic programming approach proposed by Baker and Giancarlo.

Rate: The rate of numbers of fragments between by using the A^* algorithm and the dynamic programming approach.

Table 4-1 Experimental results

S_1	S_2	Len_1	Len_2	T	M	A^*	DP	$Rate$
HT	A4	663	666	7	74	8	74	10.81%
pAD14	HMA	666	666	7	70	16	70	22.85%
8884H	7768H	666	666	7	39	20	39	51.28%
CYP3A26	LOC489851	1925	1743	8	150	68	150	45.33%
At2g30770	At1g11610	1632	1449	8	107	72	107	67.29%
Fmo3	FMO3	2020	1805	8	140	87	140	62.14%
P0452F10.9	P0452F10.11	1590	1656	9	98	54	98	55.10%
B1131G07.28-2	B1131G07.28-1	1529	1890	9	49	49	49	100%

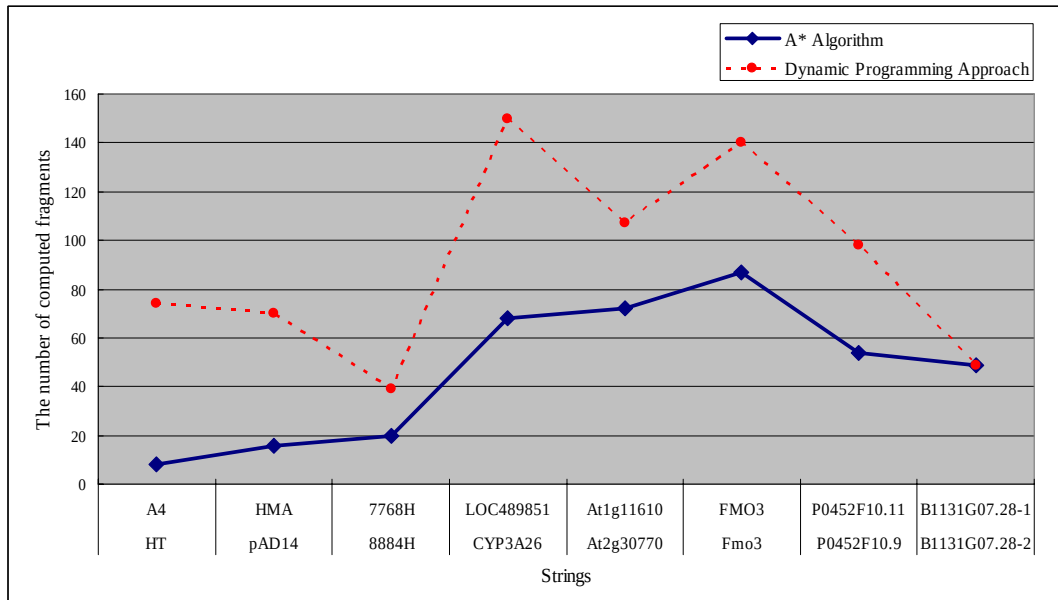


Fig. 4-1 Comparing the number of computed fragments by using the A^* algorithm and the number of computed fragments by using the dynamic programming approach.

By the above results, we conclude that the A^* algorithm is quite efficient as the number of fragments which need to be computed is much smaller than that of using the dynamic programming approach in general. Of course, for some cases, the number of fragments, need to be computed by the A^* algorithm, may be the same as the number of fragments, need to be computed by dynamic programming approach. For instance, two strings B1131G07.28-2 and B1131G07.28-1, the rate of numbers of fragments between by using the two approaches reaches 100%. In generally, the performance of the A^* algorithm is better than the dynamic programming approach.

Chapter 5 Conclusions

In this thesis, we proposed the application of the A^* algorithm to solve the longest common subsequence from fragments problem. The A^* algorithm can successfully filter out some fragments which wouldn't appear in solutions, and efficiently find a solution. If given fragments are distributed over the edit graph, the method can ignore a lot fragments. The method is quite efficient as the number of fragments which are needed to be computed is smaller than the dynamic programming approach proposed in Baker and Giancarlo. In general cases, a few fragments are computed in solving the problem. However, in worst cases, all fragments are needed to be computed in solving process.

Bibliography

[ALP2004] Faster Algorithms for String Matching with k Mismatches, Amir, A., Lewenstein, M. and Porat, E., Journal of Algorithms, Vol. 50, 2004, pp. 257-275.

[BG2002] Sparse Dynamic Programming for Longest Common Subsequence from Fragments, Baker, B. S. and Giancarlo, R., Journal of Algorithms, Vol. 42, 2002, pp. 231-254.

[BLP97] Approximation Algorithms for Multiple Sequence Alignment, Bafna, V., Lawler, E. L. and Pevzner, P. A., Theoretical Computer Science, Vol. 182, 1997, pp. 233-244.

[BM77] A Fast String Searching Algorithm, Boyer, R. S. and Moore, J. S., Communication of the ACM, Vol. 20, 1977, pp. 762-772.

[CL88] The Multiple Sequence Alignment problem in Biology, Carrillo, H. and Lipman, D. J., SIAM Journal on Applied Mathematics, Vol. 48, 1988, pp. 1073-1082.

[CR2002] Jewels of Stringology, Crochemore, M. and Rytter, W., World Scientific, 2002.

[H75] A Linear Space Algorithm for Computing Maximal Common Subsequences, Hirschberg, D. S., Communications of the ACM, Vol. 18, No. 6, 1975, pp. 341-343.

- [HS77] A Fast Algorithm for Computing Longest Common Subsequences, Hunt, J. W. and Szymanski, T. G., Communications of the ACM, Vol. 20, No. 5, 1977, pp. 350-353.
- [J82] A Priority Queue in which Initialization and Queue Operations Take $O(\log \log D)$ time, Johnson, D. B., Mathematical Systems Theory, Vol. 15, 1982, pp. 295-309.
- [KMP77] Fast Pattern Matching in Strings, Knuth, D., Morris, J. and Pratt, V., SIAM Journal on Computing, Vol. 6, 1977, pp. 323-350.
- [KS83] Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison, Kruskal, D. B. and Sankoff, D. (Eds.), Addison-Wesley, Reading, MA, 1983.
- [L66] Binary Codes Capable of Correcting Deletions, Insertions, and Reversals, Levenshtein, V. I., Soviet Physics Doklady, Vol. 10, 1966, pp. 707-710.
- [LCTT91] Introduction to the Design and Analysis of Algorithms, Lee, R. C. T., Chang, R. C., Tseng, S. S. and Tsai, Y. T., Flag Corporation, Second Edition, ISBN:957-717-777-8, 1991.
- [M86] An $O(ND)$ Difference Algorithm and Its Variations, Myers, E. W., Algorithmica, Vol. 1, No. 2, 1986, pp. 251-266.
- [NW70] A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins, Needleman, S. B. and Wunsch, C. D., Journal of Molecular

Biology, Vol. 48, 1970, pp. 443-453.

[SW81] Identification of common Molecular Subsequences, Smith, T. F. and Waterman, M. S., Journal of Molecular Biology, Vol. 147, 1981, pp. 195-197.

[ZBM98] Alignments Without Low-Scoring Regions, Zhang, Z., Berman, P. and Miller, W., Research in Computational Molecular Biology (RECOMB), Vol. 5, 1998, pp. 294-301.